Formalizing Quantitative Equational Reasoning

Benjamin Bennetzen

Program Verification Exam • 07/09/2026

Aalborg University • bcb@aau.dk

Supervised by Giorgio Bacci and Robin Kaarsgaard

- Equations are fundamental to mathematical reasoning.
- Reasoning about programs is also based on program equivalences.
- But exact equivalence is often too strict. An approximation, errors, probabilistic and quantum computing.
- We want quantitative analogues of equational reasoning.

$$s \quad =_{\varepsilon} \quad t$$

$$s \quad =_\varepsilon \quad t$$

"s is within ε distance of t"

Variables

$s, t, u, \ldots$

Operations

$\Sigma_{CA} = \{ +_p \mid 0 < p < 1 \},$

Equations

(Idempotency) $\quad s +_p s = s$

(Skew commutativity) $\quad s +_p t = t +_{1-p} s$

(Skew associativity) $\quad (s +_p t) +_q u = s +_{pq} \left(t +_{\frac{(1-p)q}{1-pq}} u \right)$

Also known as convex algebras or convex spaces.

src/Signature.v

```
Record signature := {
  sym : Type;
  arity : sym → nat;
}.

Inductive term (sig : signature) (X : Type) : Type :=
  | Var : X → term sig X
  | App : forall f : sym sig, ('I_(arity f) → term sig X) → term sig X.

Definition algebra_ops (sig : signature) (A : Type) : Type :=
  forall f : sym sig, (ordinal (arity f) → A) → A.
```

- `sym` is an arbitrary `Type`, so the signature may be uncountable, as Σ_{CA} is.
- Arguments are given as functions with argumnets of type ordinal `'I_n`.

A model interprets each $+_p$ while respecting the axioms.

One model: the finitely supported probability distributions $\mathcal{D}(X)$ with convex combinations

$$(\mu +_p \nu)(x) \;=\; p\,\mu(x) + (1-p)\,\nu(x).$$

A model interprets each $+_p$ while respecting the axioms.

One model: the finitely supported probability distributions $\mathcal{D}(X)$ with convex combinations

$$(\mu +_p \nu)(x) \;=\; p\,\mu(x) + (1-p)\,\nu(x).$$

Small example. $X = \{a, b, c\}$, written as vectors $(\mu(a), \mu(b), \mu(c))$. Skew associativity with $p = q = \frac{1}{2}$ (so $pq = \frac{1}{4}$ and $\frac{(1-p)q}{1-pq} = \frac{1}{3}$):

$$\underbrace{(\delta_a +_{\frac{1}{2}} \delta_b) +_{\frac{1}{2}} \delta_c}_{(\frac{1}{4}, \frac{1}{4}, \frac{1}{2})} \;=\; \underbrace{\delta_a +_{\frac{1}{4}} (\delta_b +_{\frac{1}{3}} \delta_c)}_{(\frac{1}{4}, \frac{1}{4}, \frac{1}{2})}$$

Formalization: distributions and convex mixtures

```
Record probability_weight (R : realType) := {
  weight : R;   weight_unit : (0 ≤ weight ≤ 1)%R;
}.
Record probability_distribution (R : realType) (X : finType) := {
  probability_mass :> X → R;
  probability_mass_ge0 : forall x, (0 ≤ probability_mass x)%R;
  probability_mass_total : \sum_(x : X) probability_mass x = 1;
}.
Definition weighted_sum (p : probability_weight R) (x y : R) : R :=
  weight p * x + (1 - weight p) * y.

Definition convex_mixture (p : probability_weight R)
    (mu nu : probability_distribution R X) : probability_distribution R X.
Proof.
  refine {| probability_mass := fun x ⇒ weighted_sum p (mu x) (nu x) |}.
  ...
Defined.
```

- `X : finType` makes "finitely supported" automatic.
- The three barycentric axioms become `convex_mixture_idem`, `_skew_comm` and `_skew_assoc`: each is a scalar identity such as $px + (1-p)x = x$, discharged by `lra` or `field` and then lifted by `probability_distribution_ext`.

An algebraic theory has three equivalent presentations.

Quantitative equational theories
operations + quantitative equations.

EMet-Lawvere theories
Lawvere categories enriched in the category of extended metric spaces.

Monads on Set **(EMet)**
quantitative effects.

Mardare, Panangaden and Plotkin (LICS'16, LICS'17);
Bacci, Mardare, Panangaden and Plotkin.

If $s =_\varepsilon t$, then for an interpretation $[\![\cdot]\!]$ we ask

$$d\big([\![s]\!], [\![t]\!]\big) \;\leq\; \varepsilon,$$

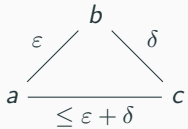
where d is a metric.

If $s =_\varepsilon t$, then for an interpretation $[\![\cdot]\!]$ we ask

$$d\big([\![s]\!], [\![t]\!]\big) \;\leq\; \varepsilon,$$

where d is a metric.

$$\frac{a =_\varepsilon b \qquad b =_\delta c}{a =_{\varepsilon+\delta} c}$$



Since d is a metric: The triangle inequality makes the rule above sound, and $d(a, b) = 0 \Rightarrow a = b$ makes $s =_0 t$ mean strict equality.

Relaxing to fuzzy relations

A generalization relaxes the metric requirement to simply a fuzzy relation

$$d : A \times A \longrightarrow [0, 1].$$

No additional assumptions: no symmetry, triangle inequality, or $d(a, a) = 0$.

Relaxing to fuzzy relations

A generalization relaxes the metric requirement to simply a fuzzy relation

$$d : A \times A \longrightarrow [0, 1].$$

No additional assumptions: no symmetry, triangle inequality, or $d(a, a) = 0$.

Some caveats with this generalization:

- The "transitivity" rule becomes unsound, and so do quantitative reflexivity and symmetry.
- We need $=$ as a separate judgement form denoting exact equality, since $s =_0 t$ does not imply $[\![s]\!] = [\![t]\!]$.

Mio, Sarkis and Vignudelli, LMCS 20 (2024).

Formalization: fuzzy spaces and algebras

```
Record fuzzy_space (R : realType) := {
    fcarrier : Type;
    frel : fcarrier → fcarrier → R;
    frel_range : forall x y, (0 ≤ frel x y ≤ 1)%R;
}.

Definition frel_nonexpansive {R : realType} (A B : fuzzy_space R)
    (f : fcarrier A → fcarrier B) : Prop :=
  forall x y, (frel B (f x) (f y) ≤ frel A x y)%R.

Record FuzzyAlgebra (R : realType) (sig : signature) := {
  fa_space : fuzzy_space R;
  fa_ops : algebra_ops sig (fcarrier fa_space)
}.
```

Let $\mathbb{A} = (A, \{op^{\mathbb{A}}\}, d_{\mathbb{A}})$ be a quantitative algebra.

$$\mathbb{A} \models \forall (B, d_B).\, s = t \quad \text{iff} \quad [\![s]\!] = [\![t]\!]$$

$$\mathbb{A} \models \forall (B, d_B).\, s =_\varepsilon t \quad \text{iff} \quad d_{\mathbb{A}}\big([\![s]\!], [\![t]\!]\big) \leq \varepsilon$$

for all non-expansive $[\![\cdot]\!] : (B, d_B) \to (A, d_{\mathbb{A}})$.

Let $\mathbb{A} = (A, \{op^{\mathbb{A}}\}, d_{\mathbb{A}})$ be a quantitative algebra.

$$\mathbb{A} \models \forall (B, d_B).\, s = t \quad \text{iff} \quad [\![s]\!] = [\![t]\!]$$

$$\mathbb{A} \models \forall (B, d_B).\, s =_\varepsilon t \quad \text{iff} \quad d_{\mathbb{A}}\big([\![s]\!], [\![t]\!]\big) \leq \varepsilon$$

for all non-expansive $[\![\cdot]\!] : (B, d_B) \to (A, d_{\mathbb{A}})$.

The context (B, d_B) is itself a fuzzy relation on the variables. It restricts which assignments are admissible.

Formalization: judgements and interpretations

```
Inductive judgement {R : realType}
    (sig : signature) (X : Type) :=
  | EqJ  : term sig X → term sig X →
           judgement sig X
  | QEqJ : R → term sig X → term sig X →
           judgement sig X.
```

```
Record interpretation
    {R : realType} {sig}
    (X : fuzzy_space R)
    (A : FuzzyAlgebra R sig) := {
  interpretation_fun :>
    fcarrier X → fcarrier (fa_space A);
  interpretation_nexp :
    frel_nonexpansive X (fa_space A)
      interpretation_fun;
}.
```

```
Fixpoint fuzzy_eval {R : realType} {sig X} (A : FuzzyAlgebra R sig)
    (rho : X → fcarrier (fa_space A)) (t : term sig X) :
    fcarrier (fa_space A) :=
  match t with
  | Var x ⇒ rho x
  | App f args ⇒ fa_ops A f (fun i ⇒ fuzzy_eval A rho (args i))
  end.
```

The two judgement forms of the previous slide are the two constructors.

Formalization: satisfaction, theories, models

```
Definition satisfies {R : realType} {sig} (A : FuzzyAlgebra R sig)
    (X : fuzzy_space R) (phi : judgement sig (fcarrier X)) : Prop :=
  forall rho : interpretation X A,
    match phi with
    | EqJ s t ⇒ fuzzy_eval A rho s = fuzzy_eval A rho t
    | QEqJ eps s t ⇒
      (frel (fa_space A) (fuzzy_eval A rho s) (fuzzy_eval A rho t) ≤ eps)%R
    end.

Definition fuzzy_theory (R : realType) (sig : signature) :=
  forall X : fuzzy_space R, @judgement R sig (fcarrier X) → Prop.

Definition fuzzy_models {R : realType} {sig} (A : FuzzyAlgebra R sig)
    (E : fuzzy_theory R sig) : Prop :=
  forall X phi, E X phi → satisfies A X phi.
```

`satisfies` is $\models$ for a single judgement and `fuzzy_models` is $\mathbb{A} \models E$.

$$\text{(Idempotency)} \quad \forall \begin{pmatrix} & x \\ x & \begin{bmatrix} 1 \end{bmatrix} \end{pmatrix} . \; x +_p x = x$$

$$\text{(Skew comm.)} \quad \forall \begin{pmatrix} & x & y \\ x & \begin{bmatrix} 1 & 1 \end{bmatrix} \\ y & \begin{bmatrix} 1 & 1 \end{bmatrix} \end{pmatrix} . \; x +_p y = y +_{1-p} x$$

$$\text{(Skew assoc.)} \quad \forall \begin{pmatrix} & x & y & z \\ x & \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \\ y & \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \\ z & \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \end{pmatrix} . \; (x +_p y) +_q z = x +_{pq} \left(y +_{\frac{(\mathbf{1}-p)q}{\mathbf{1}-pq}} z \right)$$

$$\text{(Interpolative)} \quad \forall \begin{pmatrix} & x & y & w & z \\ x & \begin{bmatrix} 1 & 1 & \varepsilon & 1 \end{bmatrix} \\ y & \begin{bmatrix} 1 & 1 & 1 & \delta \end{bmatrix} \\ w & \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \\ z & \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \end{pmatrix} . \; x +_p y \; =_{p\varepsilon + (1-p)\delta} \; w +_p z$$

The first three are the old axioms over the discrete context. The fourth constrains only $d([\![x]\!], [\![w]\!]) \leq \varepsilon$ and $d([\![y]\!], [\![z]\!]) \leq \delta$; every other entry is 1. (the theory $\mathbb{ICA}$)

Formalization: the $\mathbb{ICA}$ signature and its contexts

```
Record ica_weight (R : realType) := {
  ica_weight_val : R;
  ica_weight_open : (0 < ica_weight_val < 1)%R;   (* strictly inside (0,1) *)
}.
Inductive ica_sym (R : realType) : Type :=
  | ica_plus : ica_weight R → ica_sym R.
Definition ica_signature {R : realType} : signature :=
  {| sym := ica_sym R; arity := fun _ ⇒ 2 |}.
Notation "x <+ p +> y" := (ica_op p x y).
```

```
Definition ica_full_space (R : realType) (n : nat) : fuzzy_space R.
  (* 'I_n with frel = fun _ _ ⇒ 1 : the discrete fuzzy relation *)

Definition ica_interp_rel {R : realType} (eps delta : R) (a b : 'I_4) : R :=
  match nat_of_ord a, nat_of_ord b with
  | 0, 2 ⇒ eps  | 1, 3 ⇒ delta  | _, _ ⇒ 1
  end.
```

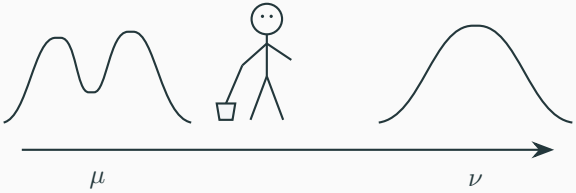
Under $0 = x$, $1 = y$, $2 = w$, $3 = z$ the second one is the interpolative matrix, verbatim.

Formalization: `Inductive ica_theory`

```
Inductive ica_theory (R : realType) : fuzzy_theory R (@ica_signature R) :=
  | ICA_Idem (p : ica_weight R) :
      @ica_theory R (ica_full_space R 1)
        (EqJ ((Var (inord 0)) <+ p +> (Var (inord 0))) (Var (inord 0)))
  | ICA_Skew_Comm (p : ica_weight R) :
      @ica_theory R (ica_full_space R 2)
        (EqJ ((Var (inord 0)) <+ p +> (Var (inord 1)))
             ((Var (inord 1)) <+ ica_weight_complement p +> (Var (inord 0))))
  | ICA_Skew_Assoc (p q : ica_weight R) :
      @ica_theory R (ica_full_space R 3)
        (EqJ (((Var (inord 0)) <+ p +> (Var (inord 1))) <+ q +> (Var (inord 2)))
             ((Var (inord 0)) <+ ica_weight_product p q +>
               ((Var (inord 1)) <+ ica_weight_assoc_inner p q +> (Var (inord 2)))))
  | ICA_Interp (p : ica_weight R) (eps delta : R)
      (Heps : (0 ≤ eps ≤ 1)%R) (Hdelta : (0 ≤ delta ≤ 1)%R) :
      @ica_theory R (@ica_interp_space R eps delta Heps Hdelta)
        (QEqJ (weighted_sum (ica_probability_weight p) eps delta)
          ((Var (inord 0)) <+ p +> (Var (inord 1)))
          ((Var (inord 2)) <+ p +> (Var (inord 3)))).
```

An `Inductive` in `fuzzy_theory R sig`, one constructor per axiom scheme.

An aside about moving piles of sand



Out of all the possible ways to move the sand, which is the least amount of work? (Also known as optimal transport)

$$\mathcal{K}(d)(\mu,\nu) = \inf_{\gamma\in\Gamma(\mu,\nu)} \underbrace{\sum_{a,b\in A} \gamma(a,b)\cdot d(a,b)}_{V_d(\gamma)}$$

$\Gamma(\mu,\nu)$ is the set of joint distributions with marginals μ and ν, the transport plans.

Formalization: couplings and the lifting

```
Record coupling {R : realType} {X Y : finType}
    (mu : probability_distribution R X)
    (nu : probability_distribution R Y) := {
  coupling_distribution : joint_probability_distribution R X Y;
  coupling_fst : forall x, \sum_(y : Y) coupling_distribution x y = mu x;
  coupling_snd : forall y, \sum_(x : X) coupling_distribution x y = nu y;
}.

Definition coupling_cost {R : realType} {X Y : finType} (d : X → Y → R)
    {mu} {nu} (gamma : coupling mu nu) : R :=
  \sum_(x : X) \sum_(y : Y) coupling_distribution gamma x y * d x y.

Definition kantorovich_lifting {R : realType} {X : finType} (d : X → X → R)
    (mu nu : probability_distribution R X) : R :=
  inf [set r : R | exists gamma : coupling mu nu, r = coupling_cost d gamma].
```

- A `coupling` is a joint distribution bundled with its two marginal equations.

The finitely supported distributions, their convex sums, and the Kantorovich lifting form a quantitative algebra $(\mathcal{D}(A), \{+_p\}, \mathcal{K}(d))$.

The finitely supported distributions, their convex sums, and the Kantorovich lifting form a quantitative algebra $(\mathcal{D}(A), \{+_p\}, \mathcal{K}(d))$.

Adding the interpolative axiom is exactly what axiomatises it:

Soundness (Prop. 4.6 and Cor. 4.7)

If $s =_\varepsilon t$ is derivable, then $\mathcal{K}(d)([\![s]\!], [\![t]\!]) \leq \varepsilon$.

Completeness (Prop. 4.10)

If $\mathcal{K}(d)([\![s]\!], [\![t]\!]) \leq \varepsilon$, then $s =_\varepsilon t$ is derivable.

Mio, Compact Quantitative Theories of Convex Algebras, MFPS 2025.

Formalization: the Kantorovich algebra

```
Section KantorovichModel.
Context {R : realType} {X : finType}.
Variable d : X → X → R.
Hypothesis Hd : forall x y, (0 ≤ d x y ≤ 1)%R.

Definition finite_fuzzy_space : fuzzy_space R :=
  {| fcarrier := X; frel := d; frel_range := Hd |}.

Definition kantorovich_fuzzy_space : fuzzy_space R :=
  {| fcarrier := probability_distribution R X;
     frel := kantorovich_lifting d;
     frel_range := fun mu nu ⇒ fuzzy_kantorovich_lifting mu nu Hd |}.

Definition kantorovich_algebra : FuzzyAlgebra R (@ica_signature R) :=
  {| fa_space := kantorovich_fuzzy_space; fa_ops := kantorovich_ops |}.
```

`kantorovich_ops` interprets `ica_plus p` as `convex_mixture`, and `fuzzy_kantorovich_lifting` is the bound $0 \leq \mathcal{K}(d) \leq 1$, which is the range obligation that makes $\mathcal{K}(d)$ a legal `fuzzy_space`. So this record is $(\mathcal{D}(A), \{+_p\}, \mathcal{K}(d))$, and the goal is now `fuzzy_models kantorovich_algebra ica_theory`.

The proof idea: run both transport plans in parallel

Assume $\mathcal{K}(d)(\mu_x, \mu_w) \leq \varepsilon$ and $\mathcal{K}(d)(\mu_y, \mu_z) \leq \delta$. We must show

$$\mathcal{K}(d)(\mu_x +_p \mu_y,\ \mu_w +_p \mu_z) \ \leq\ p\,\varepsilon + (1-p)\,\delta.$$

The proof idea: run both transport plans in parallel

Assume $\mathcal{K}(d)(\mu_x, \mu_w) \le \varepsilon$ and $\mathcal{K}(d)(\mu_y, \mu_z) \le \delta$. We must show

$$\mathcal{K}(d)(\mu_x +_p \mu_y,\ \mu_w +_p \mu_z) \ \le\ p\,\varepsilon + (1-p)\,\delta.$$

1. $\mathcal{K}$ is an infimum, so take almost optimal plans: $\gamma \in \Gamma(\mu_x, \mu_w)$ and $\gamma' \in \Gamma(\mu_y, \mu_z)$ with $V_d(\gamma) \le \varepsilon + \lambda$, $V_d(\gamma') \le \delta + \lambda$.
2. $\gamma +_p \gamma' \in \Gamma(\mu_x +_p \mu_y,\ \mu_w +_p \mu_z)$: run the first plan on the p-fraction of the mass and the second on the rest.
3. V_d is "mixture-preserving": $V_d(\gamma +_p \gamma') = p\,V_d(\gamma) + (1-p)V_d(\gamma')$.
4. The infimum is below this one plan, so the bound is $p\varepsilon + (1-p)\delta + \lambda$; then let $\lambda \to 0$.

The lemmas the proof needs

```
Lemma coupling_convex_mixture (p : probability_weight R)
    (gamma1 : coupling mu1 nu1) (gamma2 : coupling mu2 nu2) :
  coupling (convex_mixture p mu1 mu2) (convex_mixture p nu1 nu2).
```

```
Lemma kantorovich_almost_optimal (d : X → X → R) (mu nu : ...) (eta : R) :
  (forall x y, (0 ≤ d x y)%R) → (0 < eta)%R →
  exists gamma : coupling mu nu,
    (coupling_cost d gamma < kantorovich_lifting d mu nu + eta)%R.

Lemma coupling_cost_convex_mixture (p : probability_weight R) ... :
  coupling_cost d (coupling_convex_mixture p gamma1 gamma2)
    = weighted_sum p (coupling_cost d gamma1) (coupling_cost d gamma2).

Lemma kantorovich_convex_mixture (d : X → X → R) (p : probability_weight R) :
  (forall x y, (0 ≤ d x y ≤ 1)%R) →
  (kantorovich_lifting d (convex_mixture p mu1 mu2) (convex_mixture p nu1 nu2)
     ≤ weighted_sum p (kantorovich_lifting d mu1 nu1)
                      (kantorovich_lifting d mu2 nu2))%R.
```

Modelhood: the three equational goals

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                     SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.                    SkewAssoc
  - rewrite !kantorovich_eval_op.                    Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

Goal 1 / 4 (ICA_Idem)

```
d : X → X → R
Hd : forall x y : X, 0 ≤ d x y ≤ 1
p : ica_weight R
rho : interpretation (ica_full_space R 1)
        kantorovich_algebra
```

```
⊢ fuzzy_eval kantorovich_algebra rho
     (Var (inord 0) <+ p +> Var (inord 0)) =
   fuzzy_eval kantorovich_algebra rho (Var (inord 0))
```

`case: Hphi` on the inductive theory gives one goal per axiom, four in total.

The first three are equalities of distributions. They are closed by the `convex_mixture_*` lemmas from earlier, one tactic each.

Modelhood: the interpolative goal

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.                SkewAssoc
  - rewrite !kantorovich_eval_op.                    Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

Goal 4 / 4 (ICA_Interp), as it arrives

```
p : ica_weight R
eps, delta : R
Heps : 0 ≤ eps ≤ 1
Hdelta : 0 ≤ delta ≤ 1
rho : interpretation (ica_interp_space Heps Hdelta)
        kantorovich_algebra
```

```
⊢ frel (fa_space kantorovich_algebra)
     (fuzzy_eval kantorovich_algebra rho
        (Var (inord 0) <+ p +> Var (inord 1)))
     (fuzzy_eval kantorovich_algebra rho
        (Var (inord 2) <+ p +> Var (inord 3)))
   ≤ weighted_sum (ica_probability_weight p) eps delta
```

The context is the four-point space carrying $(0,2) \mapsto \varepsilon$ and $(1,3) \mapsto \delta$. First: evaluate the two $+_p$-terms.

Modelhood: convexity of the lifting

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.                SkewAssoc
  - rewrite !kantorovich_eval_op.                   Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

after `rewrite !kantorovich_eval_op`

```
⊢ frel (fa_space kantorovich_algebra)
     (convex_mixture (ica_probability_weight p)
        (fuzzy_eval kantorovich_algebra rho (Var (inord 0)))
        (fuzzy_eval kantorovich_algebra rho (Var (inord 1))))
     (convex_mixture (ica_probability_weight p)
        (fuzzy_eval kantorovich_algebra rho (Var (inord 2)))
        (fuzzy_eval kantorovich_algebra rho (Var (inord 3))))
   ≤ weighted_sum (ica_probability_weight p) eps delta
```

Both sides are now literal convex mixtures, which is exactly the shape of `kantorovich_convex_mixture`.

Modelhood: split the weighted sum

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.                SkewAssoc
  - rewrite !kantorovich_eval_op.                   Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

after `apply: (le_trans (kantorovich_convex_mixture ...))`

```
⊢ weighted_sum (ica_probability_weight p)
     (kantorovich_lifting d
        (fuzzy_eval kantorovich_algebra rho (Var (inord 0)))
        (fuzzy_eval kantorovich_algebra rho (Var (inord 2))))
     (kantorovich_lifting d
        (fuzzy_eval kantorovich_algebra rho (Var (inord 1)))
        (fuzzy_eval kantorovich_algebra rho (Var (inord 3))))
   ≤ weighted_sum (ica_probability_weight p) eps delta
```

The mixture is gone: both sides are $\oplus_p$, and $\oplus_p$ is monotone in each argument.

Modelhood: non-expansiveness of `rho`

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.               SkewAssoc
  - rewrite !kantorovich_eval_op.                  Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

after `apply: weighted_sum_le` — Goal 1 / 2

```
⊢ kantorovich_lifting d
    (fuzzy_eval kantorovich_algebra rho (Var (inord 0)))
    (fuzzy_eval kantorovich_algebra rho (Var (inord 2)))
  ≤ eps
```

Goal 2 / 2

```
⊢ kantorovich_lifting d
    (fuzzy_eval kantorovich_algebra rho (Var (inord 1)))
    (fuzzy_eval kantorovich_algebra rho (Var (inord 3)))
  ≤ delta
```

One goal per component. Each is $\mathcal{K}(d)([\![x]\!], [\![w]\!]) \leq \varepsilon$, and `rho` is non-expansive by construction.

Modelhood: the context matrix, read off

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
       convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
       convex_mixture_skew_assoc.                SkewAssoc
  - rewrite !kantorovich_eval_op.                   Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

after `apply: (le_trans (interpretation_nexp rho (inord 0) (inord 2)))`

```
⊢ frel (ica_interp_space Heps Hdelta) (inord 0) (inord 2) ≤ eps
```

after `rewrite /= /ica_interp_rel`

```
⊢ match nat_of_ord (inord 0) with
  | 0 ⇒ match nat_of_ord (inord 2) with 2 ⇒ eps | _ ⇒ 1 end
  | 1 ⇒ match nat_of_ord (inord 2) with 3 ⇒ delta | _ ⇒ 1 end
  | _ ⇒ 1
  end ≤ eps
```

`!inordK` computes the two indices, the matrix returns its $(0, 2)$ entry, and the goal is `eps <= eps`.

Modelhood: the second component

```
Theorem kantorovich_models_ica :
  fuzzy_models kantorovich_algebra (@ica_theory R).
Proof.
  rewrite /fuzzy_models.
  move => Y phi Hphi.
  case: Hphi => [p | p | p q | p eps delta ...] rho.
  - by rewrite kantorovich_eval_op
      convex_mixture_idem.                          Idem
  - by rewrite !kantorovich_eval_op
      convex_mixture_skew_comm.                  SkewComm
  - by rewrite !kantorovich_eval_op
      convex_mixture_skew_assoc.                SkewAssoc
  - rewrite !kantorovich_eval_op.                  Interp
    apply: (le_trans
      (kantorovich_convex_mixture _ _ _ _ _ Hd)).
    apply: weighted_sum_le.
    - apply: (le_trans (interpretation_nexp
        rho (inord 0) (inord 2))).
      by rewrite /= /ica_interp_rel !inordK.
    - apply: (le_trans (interpretation_nexp
        rho (inord 1) (inord 3))).
      by rewrite /= /ica_interp_rel !inordK.
Qed.
```

the one remaining goal

```
⊢ kantorovich_lifting d
    (fuzzy_eval kantorovich_algebra rho (Var (inord 1)))
    (fuzzy_eval kantorovich_algebra rho (Var (inord 3)))
  ≤ delta
```

The same two tactics at $(1, 3)$, where the matrix returns δ.

So the whole interpolative axiom is: convexity of the lifting, monotonicity of $\oplus_p$, and two lookups in the context matrix.

What survives when d is an arbitrary fuzzy relation:

$$\frac{}{x =_{d_B(x,y)} y}$$

read off the context

$$\frac{\varepsilon \leq \delta \qquad a =_\varepsilon b}{a =_\delta b}$$

weakening

$$\frac{}{a =_1 b}$$

$d \leq 1$ always

$$\frac{\{\, s =_\delta t \mid \varepsilon < \delta \,\}}{s =_\varepsilon t}$$

- The bottom rule has infinite width, so proofs are no longer finite objects.
- Known as the order-completeness, Archimedean, or continuity rule.
- Proven to be not needed for barycentric algebras: $\mathbb{ICA}$ is a compact theory.

Formalization: one inductive, two modes

```
Inductive frel_derives {R : realType} {sig : signature}
    (E : fuzzy_theory R sig) : frel_derivation_mode →
    forall X : fuzzy_space R, @judgement R sig (fcarrier X) → Prop :=
  | FD_Init : forall mode X phi, E X phi → frel_derives E mode phi
  | FD_EqRefl | FD_EqSym | FD_EqTrans | FD_EqCong  (* Birkhoff, for [=] only *)
  | FD_UseVariables : forall mode X (x y : fcarrier X),
      frel_derives E mode (QEqJ (frel X x y) (Var x) (Var y))
  | FD_Subst : forall mode X Y phi (sigma : fcarrier X → term sig (fcarrier Y)),
      frel_derives E mode phi →
      (forall x y, frel_derives E mode (QEqJ (frel X x y) (sigma x) (sigma y))) →
      frel_derives E mode (subst_judgement sigma phi)
  | FD_QEqReplaceL | FD_QEqReplaceR   (* rewriting a [=] inside a [=_eps] *)
  | FD_Up : forall mode X eps delta s t, (eps ≤ delta)%R →
      frel_derives E mode (QEqJ eps s t) → frel_derives E mode (QEqJ delta s t)
  | FD_Max : forall mode X s t, frel_derives E mode (QEqJ 1 s t)
  | FD_OrderComplete : forall X eps (s t : term sig (fcarrier X)),
      (forall delta, (eps < delta)%R → frel_derives E FRelFull (QEqJ delta s t)) →
      frel_derives E FRelFull (QEqJ eps s t).
```

- `FD_Subst`'s second premise is exactly "σ is provably non-expansive".
- `FD_OrderComplete` is only allowed at `FRelFull`, so the finitary system lives inside the same inductive, and compactness is the statement that the two modes agree.

Soundness of the system: $\vdash \Rightarrow \models$

```
Theorem derives_fin_sound (A : FuzzyAlgebra R sig)
    (E : fuzzy_theory R sig) (X : fuzzy_space R)
    (phi : @judgement R sig (fcarrier X)) :
  fuzzy_models A E -> derives_fin E phi ->
  satisfies A X phi.
Proof.
  rewrite /fuzzy_models /satisfies /derives_fin.
  move => Hmodel Hderive rho.
  remember FRelFinite as mode eqn:Hmode in Hderive.
  revert Hmode.
  induction Hderive; intros Hmode.
  - apply: Hmodel X phi H rho.                    Init
  - by [].                                        EqRefl
  - symmetry. exact: IHHderive ...                EqSym
  - by rewrite (IHHderive1 ...) ...               EqTrans
  - congr fa_ops; apply ...                       EqCong
  - move => //=.                                  UseVars
    apply: (interpretation_nexp rho).
  - apply: satisfies_subst. ...                   Subst
  - rewrite (IHHderive1 ...). ...                 QEqRepL
  - rewrite -(IHHderive2 ...). ...                QEqRepR
  - apply: le_trans ... H.                        Up
  - apply: (andP (frel_range ...)).2              Max
  - discriminate Hmode.                           OrderCompl
Qed.
```

Proof state

```
Hmodel : forall X0 phi0, E X0 phi0 →
  forall rho0 : interpretation X0 A,
    match phi0 with ... end
Hderive : frel_derives E mode phi
rho : interpretation X A
```

```
⊢ mode = FRelFinite →
   match phi with
   | EqJ s t ⇒ fuzzy_eval A rho s = fuzzy_eval A rho t
   | QEqJ eps s t ⇒
     frel (fa_space A) (fuzzy_eval A rho s) (fuzzy_eval A rho t) ≤
      eps
   end
```

Soundness: the FD_UseVariables case

```
Theorem derives_fin_sound (A : FuzzyAlgebra R sig)
    (E : fuzzy_theory R sig) (X : fuzzy_space R)
    (phi : @judgement R sig (fcarrier X)) :
  fuzzy_models A E -> derives_fin E phi ->
  satisfies A X phi.
Proof.
  rewrite /fuzzy_models /satisfies /derives_fin.
  move => Hmodel Hderive rho.
  remember FRelFinite as mode eqn:Hmode in Hderive.
  revert Hmode.
  induction Hderive; intros Hmode.
  - apply: Hmodel X phi H rho.                    Init
  - by [].                                        EqRefl
  - symmetry. exact: IHHderive ...                EqSym
  - by rewrite (IHHderive1 ...) ...               EqTrans
  - congr fa_ops; apply ...                       EqCong
  - move => //=.                                  UseVars
    apply: (interpretation_nexp rho).
  - apply: satisfies_subst. ...                   Subst
  - rewrite (IHHderive1 ...). ...                 QEqReplL
  - rewrite -(IHHderive2 ...). ...                QEqReplR
  - apply: le_trans ... H.                        Up
  - apply: (andP (frel_range ...)).2              Max
  - discriminate Hmode.                           OrderCompl
Qed.
```

The rule $x =_{d_B(x,y)} y$: two variables are as far apart as the context space says.

Goal 6 / 12 (FD_UseVariables)

```
X : fuzzy_space R
x, y : fcarrier X
rho : interpretation X A
```

```
⊢ frel (fa_space A) (fuzzy_eval A rho (Var x))
                    (fuzzy_eval A rho (Var y))
   ≤ frel X x y
```

after `move => //=`

```
⊢ (frel (fa_space A) (rho x) (rho y) ≤ frel X x y) = true
```

`/=` evaluates `fuzzy_eval` at a `Var`, leaving `frel_nonexpansive` at x, y — the record's second field. So `apply: (interpretation_nexp rho)`.

Soundness for Kantorovich

```
Definition dirac_interpretation :
    interpretation finite_fuzzy_space kantorovich_algebra :=
  @Build_interpretation R (@ica_signature R)
    finite_fuzzy_space kantorovich_algebra
    (fun x : X ⇒ dirac x)
    (fun x y ⇒ kantorovich_dirac_le x y Hd).

Definition ica_term_distribution (s : term (@ica_signature R) X) :=
  fuzzy_eval kantorovich_algebra (fun x : X ⇒ dirac x) s.

Theorem finite_kantorovich_soundness
    (s t : term (@ica_signature R) X) (eps : R) :
  @derives_full R (@ica_signature R) (@ica_theory R)
    finite_fuzzy_space (QEqJ eps s t) →
  (kantorovich_lifting d (ica_term_distribution s)
                         (ica_term_distribution t) ≤ eps)%R.
Proof.
  move ⇒ H.
  apply: (derives_full_sound kantorovich_models_ica H dirac_interpretation).
Qed.
```

Three ingredients, one line: generic soundness, modelhood, and the Dirac interpretation. Dirac is not a restriction.

Completeness: what is in place

Done: Signature, FuzzyRelation, FRelDeduction (both modes sound), ProbabilityDistribution, KantorovichProperties, ICA.

In progress, BarycentricNormalization:

```
Fixpoint term_distribution (s : term (@ica_signature R) X)
  : probability_distribution R X.       (* the denotation [s] *)

Lemma term_distribution_support (s : term (@ica_signature R) X) :
  distribution_support (term_distribution s) = term_vars s.

Lemma condition_recover (mu : probability_distribution R X) (x : X) :
  (mu x < 1) → x \in distribution_support mu →
  convex_mixture (ica_probability_weight (ica_weight_clamp (mu x)))
    (dirac x) (condition mu x) = mu.
```

1. Normal forms. Iterate `condition_recover` to write every term as an n-ary convex combination $\sum_i p_i x_i$ over its support.

2. For $e(\boldsymbol{x}) = \sum_i p_i x_i$ and tuples $\boldsymbol{a}, \boldsymbol{b}$:

$$\mathbb{ICA} \vdash \forall (A, d).\, e(\boldsymbol{a}) =_{\varepsilon} e(\boldsymbol{b}), \qquad \varepsilon = \textstyle\sum_i p_i\, d(a_i, b_i),$$

   by induction on n, one instance of the interpolative axiom per step.

3. A coupling $\gamma \in \Gamma(\llbracket s \rrbracket, \llbracket t \rrbracket)$ is a recipe for rewriting s and t into the same shape $e(\boldsymbol{x})$; then (2) gives $s =_{V_d(\gamma)} t$, and `FD_Up` weakens to ε.

Extending to the quantum setting

In my PhD I want to apply this line of thinking to quantum computing: start with a theory for mixed quantum states, later a theory for quantum programs.

Why mixed states?

- They can be viewed as probability distributions over pure states.
- They are a model of the non-quantitative barycentric axioms: $\rho +_p \sigma = p\rho + (1-p)\sigma$ is again a density matrix.
- There is already a notion of quantum optimal transport:

$$T_C(\rho, \sigma) \;=\; \inf_{\Gamma \in \mathcal{C}(\rho,\sigma)} \mathrm{tr}(C\Gamma)$$

  where $\mathcal{C}(\rho, \sigma)$ are the quantum couplings, the bipartite states with marginals ρ and σ.

Should be easy, right?

- Turns out that the barycentric structure can only reveal the transportation of mass.
- Mixed states can be coupled in many different ways that transport the mass identically, but give different $\mathrm{tr}(C\Gamma)$. So the interpolative axiom, stated purely in terms of $+_p$, cannot separate them.
- Currently using a mix of graphs and linear algebra, which seems promising.

Conclusion

- Quantitative equations $s =_\varepsilon t$ replace exact equality by bounded distance. Over fuzzy relations the two judgement forms $=$ and $=_\varepsilon$ have to be kept apart, and the triangle rule has to go.
- Formalized in Rocq/MathComp: signatures and terms, fuzzy algebras, interpretations, and the deduction system in two modes, finitary and order-complete, inside a single inductive.
- Soundness is proved for both modes (`derives_fin_sound`, `derives_full_sound`), and the Kantorovich lifting on $\mathcal{D}(X)$ is proved to be a model of $\mathbb{ICA}$ (`kantorovich_models_ica`), which gives soundness of $\mathbb{ICA}$ with respect to the Kantorovich distance.
- Completeness is under way: the normalization machinery (`condition_recover`, `term_distribution_support`) is in place; the n-ary mixture lemma and the coupling argument are next.
- Goal: the same story for mixed quantum states, where the barycentric structure alone is not enough.

Thank you. Questions?

References

- R. Mardare, P. Panangaden, G. D. Plotkin. Quantitative algebraic reasoning. LICS'16.
- R. Mardare, P. Panangaden, G. D. Plotkin. On the axiomatizability of quantitative algebras. LICS'17.
- M. Mio, R. Sarkis, V. Vignudelli. Universal quantitative algebra for fuzzy relations and generalised metric spaces. LMCS 20(4:19), 2024.
- M. Mio. Compact quantitative theories of convex algebras. ENTICS, MFPS 2025.
- C. Villani. Optimal Transport: Old and New. Springer, 2009.
- G. Bacci, R. Mardare, P. Panangaden, G. D. Plotkin. Sum and tensor of quantitative effects. LMCS 20(4:9), 2024.